Vendor-Neutral Reason-And-Act Security Workflow

Why This Was Added

AWS Continuum shows where cloud security products are moving: not only scan and show dashboards, but continuously discover, prioritize with business context, validate whether issues are actually exploitable, and guide or automate remediation inside guardrails.

CloudGuard should use that product direction without depending on AWS. The implementation in this repo keeps E2E/Dokploy/DigitalOcean-compatible hosting and treats AWS, GCP, Kubernetes, IaC, and web scanners as evidence sources.

Capability Map

Phase	CloudGuard Capability	API Contract
Discovery	Existing scan jobs, external scanner connectors, and `/api/evidence` ingestion	`/api/jobs/scan`, `/api/evidence`
Contextual Prioritization	Tenant-scoped asset/business context plus risk scoring	`/api/risk/context`, `/api/risk/prioritize`
Validation	Safe sandbox/configuration validation jobs with proof payloads and TTL guardrails	`/api/validation/jobs`
Mitigation/Remediation	Learn/approve/enforce remediation actions with rollback plans and approval status	`/api/remediation/actions`
Threat Modeling	STRIDE model generation from architecture artifacts and scanner context	`/api/threat-models/stride`
GRC Evidence	Findings, validation proof, and remediation artifacts mapped to controls	`/api/evidence`, `/api/compliance/summary`

Tenant Segregation

Every new workflow table includes:

- `tenant_id`
- `user_id`
- stable workflow IDs such as `context_id`, `validation_id`, `action_id`, and `threat_model_id`

Scanners and products should pass `tenant_id` either in the request body or through the `x-cloudguard-tenant` header. This keeps the shared portal model consistent for all teams.

Scanner/Product Integration Pattern

1. Scanner runs independently on E2E, Dokploy, a worker VM, or another target runtime.
2. Scanner submits scan output to `/api/evidence`.
3. Scanner or portal submits asset context to `/api/risk/context`.
4. Portal calls `/api/risk/prioritize` to rank findings using severity plus exposure, business criticality, identity risk, and data sensitivity.
5. If proof is required, a worker creates `/api/validation/jobs` with a short TTL and sandbox-only guardrails.
6. Worker updates validation status with proof through `/api/validation/jobs/{validation_id}/sta`
7. Portal creates remediation actions through `/api/remediation/actions`.
8. A human approves/rejects the action before production-impacting automation.
9. All proof and remediation output is stored as evidence for compliance/GRC.

Validation Guardrails

Validation jobs default to a 5-minute TTL and include these defaults:

- isolated environment required
- production mutation disabled
- cleanup required
- validation proof stored separately from the original finding

For demos, the "show live demo" button should create vulnerable resources inside an isolated sandbox account/project/namespace, run the scan and validation, ingest proof into GRC, then destroy the resources after scan completion or TTL expiry.

Remediation Modes

Mode	Meaning	Production Use
`learn`	Recommend steps only	Safe default
`approve`	Prepare automation, wait for human approval	Recommended for early production
`enforce`	Allow approved automation to execute inside guardrails	Later phase after testing

The current implementation stores the workflow and approval state. Actual cloud mutation should remain disabled until each scanner team provides reviewed automation and rollback logic.

Boss Summary

CloudGuard has been aligned with the new industry direction represented by AWS Continuum, but in a vendor-neutral way. Instead of only scanning and showing findings, the platform now has the backend contracts for contextual risk prioritization, sandbox validation proof, approval-based remediation, STRIDE threat modeling, and GRC evidence mapping. Scanner workloads can still run separately, while the portal acts as the central evidence and decision layer.